Project Echo: A Benchmarking Study for Bioacoustic Classification Architecture Selection

Deakin University SIT374/SIT764 Capstone Project

Jack Montgomery-Parkes
Deakin University
s223232288@deakin.edu.au

1 Introduction

1.1 Motivation

In recent centuries, rates of species extinction have reached some of the highest levels in the history of the planet, resulting in rapid biodiversity loss and substantial degradation of ecosystems [1]. Even the core task of monitoring wildlife populations in remote ecosystems contains persistent challenges for conservation work [2]. Traditional survey methods, whether it be radio tracking, camera trapping, and manual field observation, are all time-consuming, geographically limited, and hard to scale across the kind of large and often inaccessible terrain where threatened species tend to live [3]. Tracking population changes over time in any systematic way is therefore a non-trivial task.

Passive acoustic monitoring offers one practical alternative [4]. Many of the species conservationists care most about are vocally active, and their calls are often distinctive enough to identify from a recording. Cheap autonomous recording devices can be deployed across a desired survey area, left to run unattended, and retrieved hours, days, or weeks later with a wealth of environmental audio data. The bottleneck then becomes interpretation: manually reviewing this audio is not feasible at scale, which is what makes automated species classification such an interesting and relevant problem.

1.2 Project Echo

Project Echo is a Deakin University capstone project building a tool to help conservationists monitor threatened animal species non-invasively through sound analysis [5]. The broader system is designed to take raw audio from field recordings and produce species-level classification results for a given survey area, with the goal of enabling population trend analysis without requiring

direct physical presence in the habitat. The system has two main parts: an audio ingestion and preprocessing pipeline, and a classification engine that identifies which species are present in a recording. This report focuses on the engine, specifically, the process of selecting which machine learning architecture should power it.

1.3 Scope

Four candidate architectures are evaluated under a consistent experimental framework on a shared labelled dataset, with the aim of identifying which is best suited to serve as the core ofProject Echo's engine. The evaluation looks beyond top-line accuracy, also considering macro F1 score (which gives a fairer picture when some species have far fewer samples than others), per-class breakdowns, inference speed, model size, and how well-calibrated each model's confidence is. The results here represent our final benchmarking run and are sufficient to draw clear conclusions about the relative strengths of each candidate.

2 Dataset and Audio Preprocessing

2.1 Dataset

The data used for this benchmarking study comes from a small but focused collection of audio recorded from the Otways region in Victoria, Australia. This dataset consists of 8000+ labelled audio recordings spanning 123 species, including Australian birds, frogs, mammals, and a selection of invasive fauna.

It's this inclusion of invasive species that reflects Project Echo's dual monitoring purpose in tracking threatened native populations whilst also detecting the presence of harmful predatory or destructive fauna. Each recording is stored as a separate audio file within a per-species subdirectory, with the directory name serving as the class label. It's worth noting that more common species contribute more samples than rarer or more difficult-to-record animals, thus we're dealing with an imbalanced data problem. Class sizes in the test set range from a single sample up to 130, hence why we record a macro F1 score alongside accuracy throughout this study; accuracy alone would overweight performance on more common species. The dataset is split into training and test sets using a standard 70/30 stratified split, ensuring each class appears in roughly the same proportion in both halves.

(a) Rosella *(Platycercus elegans)*

(b) Tiger Quoll *(Dasyurus maculatus)*

(c) Rose Robin *(Petroica rosea)*

(d) Cane Toad *(Rhinella marina)*

Figure 1: Representative species from the dataset.

2.2 Audio Preprocessing

The raw dataset audio files vary in format, sample rate, duration, and channel configuration. In order to ensure every file produces a feature of identical shape regardless of its origin, all clips are passed through a standardised preprocessing pipeline before any model sees them. Each file is loaded and resampled to a target sample rate of 32 kHz, then converted to mono if necessary. At this sample rate, our maximum functional frequency is roughly 16 kHz, which is high enough to cover most vocalisations whilst ceding that a higher fidelity would allow for greater model precision. Our clips are then cropped or zero-padded to a fixed length of 5 seconds, giving a consistent 160,000-sample waveform per file. This window length was chosen as a reasonable balance between capturing enough of a call to be identifiable whilst keeping the per-sample computational cost manageable.

2.3 Feature Extraction

Rather than feeding raw waveforms directly to the classification models, each clip is first converted to a log-scaled Mel spectrogram [6], a useful 2D mapping of energy distributed across frequency bands over time, with the frequency axis scaled to approximate how human hearing perceives pitch. This is a fairly non-controversial preprocessing choice in audio classification tasks, and it comes with the practical advantage of allowing image-based neural network architectures such as GhostNet and EfficientNetV2 to be applied directly to audio data by treating the spectrogram as a single-channel image.

The spectrogram is computed and the output is then converted to decibels using a log power scale, before finally being normalised per-clip and fed to the model. For the PANNs-CNN14 model, which was initially trained on raw waveforms, the audio array itself is used directly as opposed to being converted to a spectrogram first. We cover this distinction in more detail in Section 3. Computed features are cached to disk on first extraction so that repeated training runs do not require reprocessing the full dataset from scratch.

3 Candidate Architectures

This section introduces the four models included in the benchmark and explains why each was selected. The goal was to cover a range of the architecture space, from a simple classical baseline through to a large network pre-trained specifically on audio . Our results should therefore inform a meaningful trade-off decision rather than just picking the most accurate model in isolation.

3.1 Logistic Regression

If a more complex architecture cannot meaningfully beat a basic classifier, that is a fair warning something is wrong with the setup. Thus, before evaluating any of our selected model architectures, we needed to establish a simple baseline model. The baseline chosen for this task is a Logistic Regression classifier from scikit-learn, trained on mean-pooled Mel spectrogram features. Rather than passing the full 2D spectrogram to the model, each clip's spectrogram is averaged across the time axis, reducing it to a single 128-dimensional vector, to give one value per frequency band, before the vectors are then standardised for training on. Its role here is purely as a reference point and it is not a serious deployment candidate.

3.2 GhostNet

GhostNet [7] is a lightweight convolutional neural network designed with efficiency as a primary constraint. It developed as an attempt to take advantage of the natural tendency of standard convolution layers generating many feature maps with high levels of similarity by computing a smaller set of feature maps through normal convolutions, then generating the remainder using

cheap linear operations. The result is a network that produces comparable feature richness to a standard CNN at a fraction of the computational cost. The variant used here is `ghostnet_100`, loaded from the `timm` model library with ImageNet-pretrained weights and adapted for our single-channel spectrogram input. The final classification layer is replaced with one sized to the number of target classes. With 3.9 million parameters and approximately 245 million FLOPs per forward pass, GhostNet is the smallest and fastest neural model in this study by a significant margin. We included GhostNet as our efficiency-first candidate as it fits the requirements of a tool intended for real-world conservation deployments and, given its minimal footprint, it could feasibly run on modest field hardware.

3.3 EfficientNetV2

EfficientNetV2 [8] is a family of convolutional neural networks designed to be both accurate and fast to train [9]. Where earlier EfficientNet models scaled network width, depth, and input resolution together using a fixed compound coefficient, EfficientNetV2 uses smaller kernels in early layers and a progressive training strategy that starts with smaller image sizes and increases them over the course of training. This substantially reduces training time whilst still attempting to keep accuracy as high as possible. We're using a smaller variant, `efficientnetv2_rw_s`, loaded from `timm` with pretrained weights and adapted for single-channel spectrogram input. At 22.2 million parameters and approximately 4.77 billion FLOPs, it sits between GhostNet and PANNs-CNN14 in terms of size. We've included EfficientNetV2 as the accuracy-first candidate among the image-based models. EfficientNet models make a reliable backbone for pure accuracy-forward image recognition tasks, which makes it a natural choice for spectrogram-based audio classification [10, 11].

3.4 PANNs-CNN14

PANNs, or Pre-trained Audio Neural Networks, are a family of models trained on AudioSet [12], a large-scale dataset of over 2 million audio clips sourced from YouTube and covering 527 sound categories. CNN14 is one of the strongest models in the PANNs family, built from 14 convolutional layers and trained end-to-end on raw audio [13]. With 80.9 million parameters and over 21 billion FLOPs, it is by far the largest model we're using in this study. Using transfer learning [14], we avoid having to train our CNN14 model from scratch and only replace the final classification layer, fine-tuning for our specific target classes. This allows us to bring the model's existing understanding of the general structure of audio to the task and specialise it with relatively little additional training. A key difference in our experimental setup for this model is that we're feeding CNN14 raw waveforms rather than image spectrograms, as CNN14 handles its own internal feature extraction, hence why the standard spectrogram pipeline described in Section 2 is bypassed for this benchmark. This makes CNN14 the clear choice when deciding on our audio-specialist candidate. If domain-specific pre-training provides a meaningful advantage on this task, this is where that advantage should show up.

4 Experimental Setup

4.1 Benchmarking Environment

All benchmarking was conducted in a single Jupyter Notebook to ensure consistency across models. The core dependencies are PyTorch, used for all three neural network models, and scikit-learn, which provides the Logistic Regression baseline, the train-test split, and the evaluation metrics. Model architectures for GhostNet and EfficientNetV2 are sourced from the `timm` (PyTorch Image Models) library, while PANNs-CNN14 is loaded via the `panns-inference` package. Audio loading and spectrogram computation are handled by `librosa`, with a pinned version (0.9.2) to maintain compatibility with the PANNs inference library. Computational complexity is measured using `thop`. Training and inference were run on Apple Silicon (2023 M2 Max, v26.3.1 (a)) using the MPS (Metal Performance Shaders) backend for PyTorch GPU acceleration. PANNs-CNN14 is an exception; not all of its operations are supported by MPS at the time of writing, so it falls back to CPU for training and inference. This is reflected in its significantly longer training time and lower inference throughput relative to the other neural models, and should be kept in mind when interpreting the speed figures in Section 5.

4.2 Training Protocol

All three neural models are trained using the Adam [15] optimiser with a default, fixed learning rate for 50 epochs. Cross-entropy loss is used throughout; no learning rate scheduling or regularisation beyond the pre-trained weight initialisation is applied for any of our benchmarking runs. Our intent was to establish a clean baseline for architecture comparison rather than to maximise the performance of any single model. There is one deviation on this regime: CNN14's forward pass applies a sigmoid activation before returning its output, however, cross-entropy loss expects raw logits. Thus, the sigmoid output is inverted before loss is computed during training, with the same inversion applied at inference before softmax is used to produce per-class probabilities. The Logistic Regression baseline is trained in a single pass using a maximum of 2,000 iterations and parallelised across 8 CPU threads. As a baseline, all we needed was something quick, cheap, and easy.

4.3 Evaluation Metrics

The primary metrics we were concerned with evaluating were the top-1 test accuracy and macro-averaged F1 score. Given the natural class imbalance in wildlife recording datasets, macro F1 is arguably the more important of the two for this task; a species with 3 test samples is able to lend the same weight as one with 130. We also make sure to capture top-3 and top-5 accuracy in order to measure whether the correct species appeared anywhere in the model's top 3 or top 5 predictions. For cases where acoustically similar species might be realistically confused, a model that consistently ranks the correct answer second or third is behaving quite differently to one

that misses it entirely and should be treated as such. Finally, mean prediction confidence is reported separately for correct and incorrect predictions in order to gauge the confidence gap in our models' performance. Per-class precision, recall, and F1 are also captured, enabling a more granular look at where each architecture succeeds and struggles across the 123 species.

5 Results

5.1 Overall Performance

Each of our models were trained over 50 epochs, using a 70/30 train-test split on the Project Otways dataset.

Table 1: Headline classification results across all models.

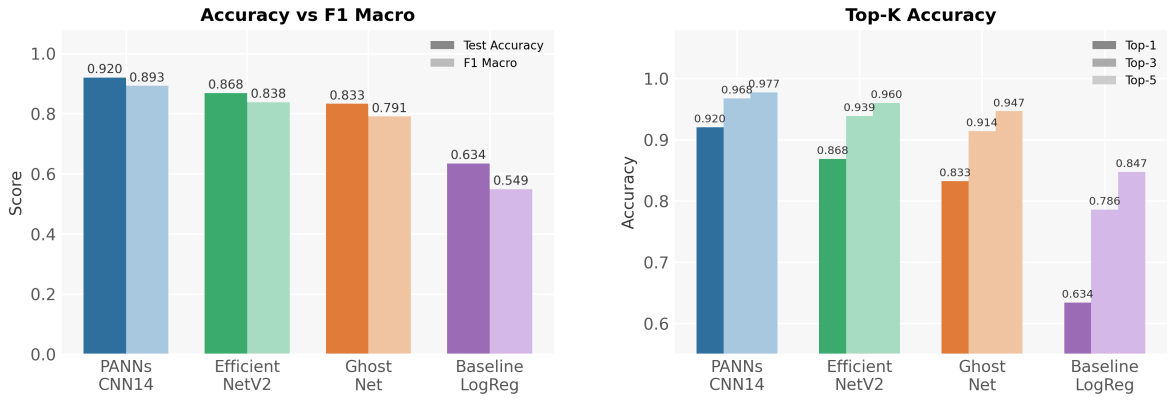
Model	Accuracy	F1 Macro	Top-3	Top-5
PANNs-CNN14	0.9203	0.8927	0.9677	0.9773
EfficientNetV2	0.8684	0.8382	0.9388	0.9600
GhostNet	0.8326	0.7906	0.9142	0.9465
Baseline-LogReg	0.6343	0.5489	0.7860	0.8472

We can see from Table 1 that PANNs-CNN14 leads on every accuracy metric at this level of training depth, hitting 92.03% top-1 accuracy with a macro F1 of 0.8927. EfficientNetV2 follows at 86.84% and 0.8382, with GhostNet close behind at 83.26% and 0.7906. As expected, the Logistic Regression baseline trails the neural models substantially.

All three neural models place the correct species in their top-3 predictions over 91% of the time, and in their top-5 over 94% of the time. Even GhostNet, our smallest model, very rarely misses the correct answer entirely.

5.2 Training Dynamics

The loss curves in Fig. 3 give an overview of how each model's training loss evolved over our 50 epochs.



(a) Test accuracy vs. F1 macro.

(b) Top-K accuracy breakdown.

Figure 2: Classification performance across all models.

Figure 3: Training loss per epoch for all neural models over 50 epochs.

Both EfficientNetV2 and GhostNet models quickly converge; EfficientNetV2 drops below 0.05 loss by epoch 5 and effectively flatlines from there, with GhostNet following a similar trajectory and reaching a similar floor around epoch 12. Both curves show some minor oscillation at low loss values, which is perhaps to be expected at this scale and learning rate. The loss curve for PANNs-CNN14 tells a decidedly different story. Its loss starts much higher and descends far more gradually, still sitting above 0.10 by epoch 50. This gives us first-hand insight into just how much

more time the fine-tuning process takes to propagate through large networks like these. That its loss is still declining at epoch 50, rather than plateauing, suggests the model had not yet fully converged and may benefit from additional training in future runs.

5.3 Confidence Calibration

Ideally our selected model should be meaningfully more confident on the predictions it gets right than on those it gets wrong; the gap between those two numbers is a practical indicator of how trustworthy the model's output is.

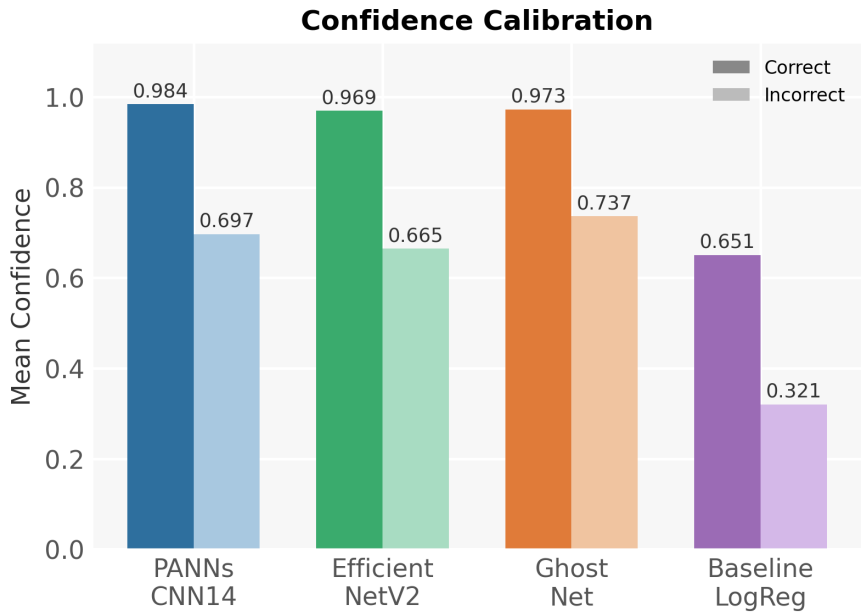


Figure 4: Mean prediction confidence for correct vs. incorrect predictions.

Looking at Fig. 4, all three of our neural models are highly confident on correct predictions, each above 0.96. EfficientNetV2 shows the largest gap between correct and incorrect confidence at 0.305, showing that it is relatively good at "knowing when it doesn't know". On the lower end of our results, GhostNet's incorrect predictions carry a slightly higher average confidence of 0.737. Whilst this isn't the most desirable calibration profile of our selected models, it's still substantially below its correct-prediction confidence.

Table 2: Speed and computational cost across all models.

Model	Train (s)	Items/s	Params	FLOPs
PANNs-CNN14	15,109	25.5	80.9M	21.0B
EfficientNetV2	3,531	313.4	22.2M	4.77B
GhostNet	1,361	712.9	3.9M	245M
Baseline-LogReg	10	12,604	—	—

The efficiency picture here is stark. From Table 2 we can see that GhostNet trains in roughly a third of EfficientNetV2's time and runs inference nearly 2.3 times faster, while carrying only 17.5% of the parameters and 5% of the FLOPs. PANNs-CNN14, running on CPU due to MPS compatibility constraints, is by far the slowest; benchmarking takes over four hours with the model only able to process 25.5 samples per second at inference. It is worth noting that this figure would improve substantially on a CUDA-capable GPU and thus should not be taken as an absolute constraint on deployment. It does, however, flag that infrastructure requirements for PANNs-CNN14 are meaningfully higher than for the other candidates.

6 Discussion

6.1 Architecture Selection

The results of this benchmarking study show that there are two viable candidates for the Project Echo Engine, each making sense under a different set of deployment priorities. PANNs-CNN14 is the strongest classifier in this study by a significant margin. Its macro F1 of 0.8927 and top-1 accuracy of 92.03% represent the best performance seen across all metrics, and its loss curve suggests it had not fully converged even at 50 epochs, suggesting there is yet more performance to be wrung from future development of the model. Where classification accuracy is paramount and sufficient compute is available, PANNs-CNN14 is by far the best architecture to take forward. On the other hand, EfficientNetV2 is the stronger candidate where practical deployment constraints may apply. At 86.84% accuracy and a macro F1 of 0.8382 it sits behind PANNs-CNN14 on raw performance, however, it trains in under an hour, processes 313 samples per second at inference, and is the only neural model to achieve non-zero F1 across all 123 test classes. For a tool intended to run on field hardware or process large recording backlogs without dedicated GPU infrastructure, EfficientNetV2 offers a compelling solution.

6.2 Limitations and Next Steps

The training protocol used throughout, a fixed learning rate with no scheduling or augmentation, was chosen for consistency rather than to maximise any individual model's performance. PANNs-CNN14 in particular may benefit from warmup scheduling or differential learning rates across its pre-trained and fine-tuned layers, and all three models are likely to improve with standard audio augmentation techniques such as time and frequency masking. While we've set the bar low during this study, with better quality and more appropriate hardware, PANNs-CNN14's speed figures could be drastically improved over current results and should not be considered as fixed for the purposes of overall model selection.

Three directions are suggested for subsequent work. First, a longer training run for PANNs-CNN14, ideally in the range of 75 to 100 epochs on appropriate (CUDA) hardware, would confirm whether its still-descending loss curve translates into further accuracy gains and clarify the true size of the gap with EfficientNetV2. Second, introducing learning rate scheduling across all models would give a cleaner picture of each architecture's ceiling [16]. Third, given that PANNs-CNN14 and EfficientNetV2 both demonstrate different strengths, an ensemble approach combining their predictions could be a promising avenue of further research. The two models have sufficiently distinct architectures and pre-training histories that they are likely to fail on different classes, which is where ensembling tends to be most effective.

7 Conclusion

Over the course of this report, we have presented a systematic benchmarking study to inform architecture selection for the Project Echo Engine, focusing specifically on four candidate models, Logistic Regression, GhostNet, EfficientNetV2, and PANNs-CNN14, on a 123-class bioacoustic classification task using a shared audio preprocessing pipeline and consistent evaluation framework. At 50 epochs, PANNs-CNN14 achieves the strongest results overall, reaching 92.03% top-1 accuracy and a macro F1 of 0.8927. Its still-descending loss curve at the end of training suggests further gains are available, making it the recommended architecture where classification performance is the priority and appropriate compute infrastructure is accessible. EfficientNetV2 is the recommended alternative where deployment constraints apply, offering 86.84% accuracy and a macro F1 of 0.8382 with substantially lower computational cost.

Both candidates represent a significant step forward from the Logistic Regression baseline, validating the case for a neural approach to the Project Echo Engine and providing a principled foundation for the next phase of development. With further training, learning rate scheduling, and an investigation into ensemble methods, there is good reason to expect performance to improve beyond what is reported here. Project Echo's broader goal of equipping conservationists with a reliable, non-invasive tool for monitoring threatened species populations through sound is well served by either of these architectures.

References

[1] S. Díaz, J. Settele, E. S. Brondízio, H. T. Ngo, J. Agard, A. Arneth, P. Balvanera, K. A. Brauman, S. H. M. Butchart, K. M. A. Chan, L. A. Garibaldi, K. Ichii, J. Liu, S. M. Subramanian, G. F. Midgley, P. Miloslavich, Z. Molnár, D. Obura, A. Pfaff, S. Polasky, A. Purvis, J. Razzaque, B. Reyers, R. R. Chowdhury, Y.-J. Shin, I. Visseren-Hamakers, K. J. Willis, and C. N. Zayas, "Pervasive human-driven decline of life on earth points to the need for transformative change." *Science*, vol. 366, no. 6471, Dec 2019.

[2] S. Gupta, P. R. Kumaresan, A. Saxena, M. Mishra, L. Upadhyay, A. Tamilmani, A. Magrey, and A. S., "Wildlife conservation and management: Challenges and strategies," *UTTAR PRADESH JOURNAL OF ZOOLOGY*, vol. 44, pp. 280–286, 12 2023.

[3] J. J. Lahoz-Monfort and M. J. L. Magrath, "A comprehensive overview of technologies for species and habitat monitoring and conservation," *BioScience*, vol. 71, no. 10, pp. 1038 – 1062, 2021, cited by: 220; All Open Access, Green Open Access, Hybrid Gold Open Access. [Online]. Available: https://www.scopus.com/inward/record.uri?eid=2-s2.0-85114188517&doi=10.1093%2fbiosci%2fbiab073&partnerID=40&md5=c407c5d935193cca016bead1af4aee98

[4] T. M. Aide, C. Corrada-Bravo, M. Campos-Cerqueira, C. Milan, G. Vega, and R. Alvarez, "Real-time bioacoustics monitoring and automated species identification," *PeerJ*, vol. 2013, no. 1, 2013, cited by: 331; All Open Access, Gold Open Access, Green Open Access. [Online]. Available: https://www.scopus.com/inward/record.uri?eid=2-s2.0-84885117413&doi=10.7717%2fpeerj.103&partnerID=40&md5=57f0714c625770cfae1344b5b1ca30b3

[5] D. Organisation, "Project echo," https://github.com/DataBytes-Organisation/Project-Echo, 2024, accessed: 2024-XX-XX.

[6] T. Bäckström, O. Räsänen, A. Zewoudie, P. P. Zarazaga, L. Koivusalo, S. Das, E. G. Mellado, M. B. Mansali, D. Ramos, S. Kadiri, P. Alku, and M. H. Vali, *Introduction to Speech Processing*, 2nd ed., 2022. [Online]. Available: https://speechprocessingbook.aalto.fi

[7] K. Han, Y. Wang, Q. Tian, J. Guo, C. Xu, and C. Xu, "Ghostnet: More features from cheap operations," 2020. [Online]. Available: https://arxiv.org/abs/1911.11907

[8] M. Tan and Q. V. Le, "Efficientnetv2: Smaller models and faster training," 2021. [Online]. Available: https://arxiv.org/abs/2104.00298

[9] ——, "Efficientnet: Rethinking model scaling for convolutional neural networks," *CoRR*, vol. abs/1905.11946, 2019. [Online]. Available: http://arxiv.org/abs/1905.11946

[10] M. Yu, D. Zhu, K. Yung, and A. W. H. Ip, "Multi-category sensitive image recognition based on refca-efficientnetv2," *International Journal of General Systems*, vol. 55, no. 3, pp. 290–314, 2026. [Online]. Available: https://doi.org/10.1080/03081079.2025.2479156

[11] J. Yao, H. Liu, and J. Ye, "Enhanced efficientnet-b0 with dual attention mechanisms for food category classification in x-ray images," *Journal of Nondestructive Evaluation*, vol. 45, 03 2026.

[12] J. F. Gemmeke, D. P. W. Ellis, D. Freedman, A. Jansen, W. Lawrence, R. C. Moore, M. Plakal, and M. Ritter, "Audio set: An ontology and human-labeled dataset for audio events," in *Proc. IEEE ICASSP 2017*, New Orleans, LA, 2017.

[13] Q. Kong, Y. Cao, T. Iqbal, Y. Wang, W. Wang, and M. D. Plumbley, "Panns: Large-scale pretrained audio neural networks for audio pattern recognition," 2020. [Online]. Available: https://arxiv.org/abs/1912.10211

[14] A. Hosna, E. Merry, J. Gyalmo, Z. Alom, Z. Aung, and M. A. Azim, "Transfer learning: a friendly introduction," *Journal of Big Data*, vol. 9, no. 1, p. 102, 2022. [Online]. Available: https://doi.org/10.1186/s40537-022-00652-w

[15] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," 2017. [Online]. Available: https://arxiv.org/abs/1412.6980

[16] M. Henheik, T. Eimer, and M. Lindauer, "Revisiting learning rate control," 2025. [Online]. Available: https://arxiv.org/abs/2507.01724